1 Lezione del 17-03-26

Nella scorsa lezione abbiamo descritto la trasformata di Fourier nella sua forma continua, in particolare esprimendo le equazioni di *analisi* e di *sintesi*. Vediamone adesso una forma numerica, che ci permette di trasformare segnali campionati arbitrati senza dover svolgere calcoli analitici.

1.1 Trasformata di Fourier discreta

Per "discretizzare" la trasformata di Fourier adottiamo il solito approccio alla discretizzazione di qualsiasi integrale, cioè preso:

$$x(t) \to \int x(t)\, dt$$

scegliamo un *intervallo di campionamento* Δt:

$$\Delta t, \quad f_s = \frac{1}{\Delta t}$$

dove f_s diventerà la nostra *frequenza di campionamento*, e portiamo l'integrale alla sommatoria:

$$\sum_{n=0}^{N-1} x(n\Delta t) \cdot \Delta t$$

su N istanti temporali (si conta da 0 a $N-1$, totale N campioni).

Applicando questo processo alla trasformata di Fourier (di cui ricordiamo l'equazione di *analisi* in 6.1), otteniamo:

$$X(k\Delta f) = \int_{-\infty}^{\infty} x(t) e^{-i2\pi k\Delta f t}\, dt \approx \Delta t \sum_{n=0}^{N-1} x(n\Delta t) e^{-i2\pi k\Delta f n\Delta t}$$

dove Δt, come abbiamo già detto, è l'intervallo di campionamento del segnale *temporale*, mentre Δf è l'intervallo di campionamento sullo spettro delle *frequenze*. Spesso si prende:

$$\Delta f = \frac{1}{N\Delta t}$$

per avere la cosiddetta frequenza "normalizzata".

Notiamo che nella formula a destra, n (indice temporale) e k (indice nelle frequenze) possono essere scelti in maniera arbitraria:

- Se calcoliamo la trasformata ad una certa frequenza $k\Delta f$, variamo k;
- Per calcolare la trasformata alla frequenza $k\Delta f$, prendiamo i contributi di tutti i possibili n fra 0 e $N-1$.

Questo significherà che potremo portare l'operazione di trasformata di Fourier discreta in forma vettoriale (è di vettori di istanti temporali o di frequenza che si parla):

$$\mathbf{X} = \begin{pmatrix} X(0) \\ \vdots \\ X((K-1)\Delta f) \end{pmatrix} = \Delta t \mathbf{F} \begin{pmatrix} x(0) \\ \vdots \\ x((N-1)\Delta t) \end{pmatrix} = \Delta t \mathbf{X}\mathbf{x}$$

dove K è la lunghezza del vettore di frequenze, $\mathbf{X}$ e $\mathbf{x}$ sono i vettori delle frequenze e dei campioni temporali rispettivamente, campionati nei punti $0, ..., K-1$ e $0, ..., N-1$.

La matrice $\mathbf{F}$ sarà invece quella degli operatori della trasformata di Fourier, cioè:

$$\mathbf{F}_{kn} = e^{-i2\pi k\Delta f n\Delta t}$$

Facciamo alcune considerazioni in termini di complessità. Abbiamo che:

- $\mathbf{X}$ ha dimensione $K \times 1$;
- $\mathbf{x}$ ha dimensione $N \times 1$;
- $\mathbf{F}$ ha dimensione $K \times N$.

Questo significherà che il numero di operazioni necessarie è nell'ordine di:

$$O(KN) \quad \left(O(N^2)\right)$$

Da cui notiamo che, visto che spesso prendiamo $K = N$, cioè tanti "bucket" (campioni) frequenziali quanti sono i campioni temporali, avremo che la complessità complessivamente sarà data da $O(N^2)$. Questo è chiaramente poco ottimale, in quanto spesso si vogliono trasformare segnali ad alte frequenze di campionamento. Abbiamo quindi bisogno di un approccio migliore.

1.1.1 Trasformata di Fourier discreta veloce

Un algoritmo più veloce per calcolare la trasformata di Fourier discreta è ad esempio quello che è stato usato per disegnare i grafici nella lezione precedente. Lo script usato per la generazione del grafico era basato sull'implementazione dell'algoritmo **FFT** (*Fast Fourier Transform*, in italiano appunto *trasformata di Fourier discreta veloce*), in particolare nella versione di *Cooley-Tukey*. Se ne trova una versione nella directory `code/python` di questi appunti.

L'algoritmo della trasformata di Fourier discreta veloce si basa sull'osservazione che negli N^2 passaggi dove combiniamo ogni valore di k con ogni valore di n si svolgono diverse volte calcoli che potrebbero essere fatti una volta sola, e quindi memoizzati.

Presentiamo quindi una versione specifica della trasformata di Fourier veloce, data dall'algoritmo di *Cooley-Tukey*, in versione a *radice 2*.

La forma della trasformata discreta che adottiamo è quella a frequenza normalizzata, cioè preso $\Delta f = \frac{1}{N\Delta t}$:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-\frac{2\pi i}{N}nk}$$

L'intuizione sarà quindi quella di dividere la DFT di dimensione N in due DFT "interlacciate" di dimensione $\frac{N}{2}$, in modo da poter risparmiare calcoli ripetuti adottando un approccio *divide et impera*. Dal punto di vista algebrico, separiamo gli indici pari e dispari:

$$X_k = \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N}(2m)k} + \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N}(2m+1)k}.$$

ed estraiamo uno degli esponenziali a destra:

$$X_k = \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2}mk} + e^{-\frac{2\pi i}{N}k} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2}mk}.$$

Possiamo quindi definire le due forme, che troviamo sostanzialmente equivalenti:

$$E_k = \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2} mk}, \quad O_k = \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2} mk}.$$

dove E_k è la trasformata di Fourier calcolata sui campioni "dimezzati" pari, e O_k è la sua analoga calcolata sui campioni dispari. Potremo quindi scrivere semplicemente:

$$X_k = E_k + e^{-\frac{2\pi i}{N} k} O_k,$$

cioè abbiamo un modo per riportare i risultati intermedi (calcolati su finestre dimezzate) nel risultato finale. Notiamo che la sfortuna di questo approccio è che vale solo per $n = 0, ..., \frac{N}{2} - 1$ compreso. Per calcolare il resto dei campioni frequenziali (da $n = \frac{N}{2}, ..., N-1$), sfruttiamo le proprietà di periodicità dell'esponenziale:

$$X_{k+N/2} = E_k - e^{-\frac{2\pi i}{N} k} O_k.$$

Notiamo che i fattori:

$$e^{-\frac{2\pi i}{N} k}, \quad -e^{-\frac{2\pi i}{N} k}$$

vengono detti *fattori di twiddle*, ed è solitamente utile precalcolarli.

Questo costituisce il nucleo dell'algoritmo FFT radice 2. Possiamo applicare l'operazione di divisione in 2 finestre "dimezzate" un numero arbitrario di volte (anche se in casi reali a conviene fermarsi prima), scendendo ricorsivamente fino al caso di dimensione 1 (dove restituiamo l'unico campione dato).

Questo, in codice Python, ha il seguente (semplicissimo) aspetto:

```python
def fft(x):
    ln = len(x)
    hln = ln // 2

    if ln == 1:
        # base case
        return x
    else:
        # calculate even/odd FFTs
        even = fft(x[0::2])
        odd  = fft(x[1::2])

        # merge even/odd FFTS in result
        res = [None] * ln
        for k in range(0, hln):
            twid = cmath.exp(-2j * math.pi * k / ln)
            X[k]       = even[k] + twid * odd[k]
            X[k + hln] = even[k] - twid * odd[k]
        return res
```

1.1.2 Antitrasformata di Fourier discreta

L'antitrasformata discreta di Fourier si calcola in maniera completamente duale alla trasformata. Ad esempio, prendendo la forma matriciale di prima, abbiamo che si passa alla matrice *hermitiana*:

$$\mathbf{x} = \Delta f \mathbf{F}^H \mathbf{X}$$

con $\mathbf{x}$, $\mathbf{F}$ e $\mathbf{X}$ presi come prima.

Anche la antitrasformata discreta ha un algoritmo ottimizzato, con intuizione e pseudocodice simile al precedente, che non riportiamo per semplicità. Tale algoritmo prende il nome di **IFFT** (*Inverse Fast Fourier Transform*).

1.2 Teoremi sulla trasformata di Fourier

Veniamo a vedere alcuni teoremi relativi alla trasformata di Fourier.

1.1: Linearità della trasformata di Fourier

Vale che:

$$x(t) = \alpha_1 x_1(t) + \alpha_2 x_2(t) \implies X(f) = \alpha_1 X_1(f) + \alpha_2 X_2(f)$$

1. Questo viene direttamente dalla forma dell'equazione di analisi:

$$X(f) = \int_{-\infty}^{\infty} \alpha_1 x_1(t) e^{-i2\pi ft}\, dt + \int_{-\infty}^{\infty} \alpha_2 x_2(t) e^{-i2\pi ft}\, dt$$

e applicando la linearità dell'operatore integrale (possiamo estrarre gli α_i), per cui:

$$X(f) = \alpha_1 X_1(f) + \alpha_2 X_2(f)$$

che è la tesi. □

2. **1.2: Dualità della trasformata di Fourier**

Vale che:

$$x(t) \to X(f) \implies X(t) \Rightarrow x(-f)$$

Questo viene in qualche modo dalle osservazioni sulla simmetria fatte in 6.1.1. In particolare, scambiando t ed f nell'equazione di sintesi, si ha:

$$x(f) = \int_{-\infty}^{\infty} X(t) e^{i2\pi t f} \, dt$$

per cui notiamo che dobbiamo scambiare f con $-f$ per ottenere:

$$x(-f) = \int_{-\infty}^{\infty} X(t) e^{-i2\pi f t} \, dt$$

che è la tesi. □

Applicando questo teorema, ad esempio, ala funzione rettangolare (vista sempre in 6.1.1), otteniamo:

$$x(t) = \text{rect}\left(\frac{t}{\tau}\right) = \tau \text{sinc}(f\tau) \implies \tau \text{sinc}(t\tau) = \text{rect}\left(\frac{f}{\tau}\right)$$